Probe FelCloud first, then cut every call

The highest-value EdNova upgrades are cheap and deterministic, and Monday's FelCloud test decides about half of them. The most urgent item is a possible sampler bug. In two mirrored versions of LiteLLM's `ollama_chat` provider, OpenAI `frequency_penalty` is copied unscaled into Ollama's multiplicative `repeat_penalty`. If FelCloud does the same, the engine's `frequency_penalty: 0.3` reaches the model as `repeat_penalty: 0.3`, a value below 1 that rewards repetition and could explain part of the 4-8% loop rate (LiteLLM ollama_chat mirror). After that, the biggest gains for the judged metrics come from shrinking and skipping calls, not from adding smarter models. Classify latency of 2.6-5 s is probably dominated by output tokens on CPU, so a terser classify plus a confidence-gated rule shortcut should move requests per turn from about 2.08 toward about 1.6 (an estimate). The other first-tier upgrades are: separating the three kinds of 429 (the observed `Retry-After: 5` matches LiteLLM's shared deployment cooldown, not EdNova's key quota), client-side token counting with a truncation canary, cache-key hygiene plus canonical normalization, and a sympy "fact sheet" that grounds every formula the LLM writes. Several fashionable techniques don't fit a 2,048-token, 20-requests-per-minute, CPU-only budget: LLMLingua, RM3, SPLADE, sampling-based hallucination detectors, trained routers and the Moroccan Atlas-Chat models all belong on the skip list. This report starts with the Monday probes, then gives a tiered plan sized for a two-person team before the 10 November Phase 1 deadline, then the skip list, then a list of claims that are still unverified.

Legend used throughout. Judged metrics: **OS** = orchestration split, **CH** = constraint handling, **RS** = resilience, **RR** = repeated requests, **RL** = relevance and grounding. Effort is in person-days (pd). "Compact" means the <3B tier (llama3.2-3b, qwen2.5-3b). "Mid-size" means the ⩽16B tier (llama3.1-8b today). ⚠ marks a claim that is unverified, comes from a secondary source, or is not peer-reviewed (see the list near the end).

Fifteen Monday probes decide which upgrades to build

Run these probes at background priority, at no more than 15 requests per minute. Log the raw status, body and headers of every response. Together they cost about 250-400 requests, roughly 15-20 minutes of full quota spread over the day. Before deliberately provoking any 429, ask whether Monday traffic counts toward the judges' per-key refusal metrics. Never try to trigger a deployment cooldown: LiteLLM applies cooldown per deployment, so other teams would be locked out too (LiteLLM routing).

#	Probe	How to run it	How to read it → what it decides
P1	Model inventory	`GET /v1/models`. Look for Qwen3-4B-Instruct-2507, Gemma-3-4B-IT, qwen2.5-7b, any Tunisian model (ESPRIT-Derja, Labess). Ask the organizers which provider prefix each model uses (`ollama/`, `ollama_chat/`, `openai/`, `hosted_vllm/`).	Decides model routing (T2-6) and startup name validation (T1-5). The prefix tells you whether OpenAI or Ollama parameter semantics apply.
P2	`frequency_penalty` mapping	On a compact model, use a long "list and explain the derivative rules" prompt at temperature 0.7, max_tokens 250, with 15 fixed seeds. Run three arms: fp omitted, fp = 0.3, fp = 1.05.	If fp = 0.3 loops more than "omitted" while fp = 1.05 looks normal, fp is being mapped to `repeat_penalty`. If fp = 1.05 produces garbled text that avoids reusing words (even "x"), the backend uses true OpenAI semantics. Decides T1-1.
P3	Extra-body passthrough and backend identity	Send extreme values in `extra_body` so "applied" and "silently dropped" look different: `{"top_k":1}` at temperature 1.0 across 3 seeds (identical outputs = applied); `{"repeat_penalty":2.0}` (garbled = applied); `{"dry_multiplier":5,"dry_allowed_length":1}` (garbled = applied, which means a llama.cpp backend). Record any error text.	Decides T2-3 (DRY, min_p, repeat_penalty). LiteLLM raises an error on unsupported params unless `drop_params` is on, in which case it drops them silently (LiteLLM drop_params).
P4	Logprob passthrough	Send `logprobs: true, top_logprobs: 5` to both tiers. Check whether `choices[0].logprobs.content`	Non-null → T2-2. Null → drop logprob

#	Probe	How to run it	How to read it → what it decides
		is non-null, and whether "3" and " 3" are different tokens.	ideas entirely.
P5	`json_schema` support	Send `response_format: {"type":"json_schema", ...}` with an enum `["alpha","beta","gamma"]` and a prompt that pushes toward answering "delta". Repeat on mistral-7b and qwen2.5-coder-7b and save the exact error text.	Output in the enum → constrained decoding is applied → T2-1. "delta" in the output → the schema is ignored. An UnsupportedParams error on the 7B models means a proxy configuration problem, not a model weakness.
P6	LiteLLM response cache	Send an identical request twice. Look for an `x-litellm-cache-key` header and check whether `x-ratelimit-api_key-remaining-requests` drops on the second call (LiteLLM caching).	Header present → T2-9. Either way, EdNova's own cache stays the source of truth.
P7	The three kinds of 429	Once each, at a quiet time, with max_tokens = 1: send a 21st request inside one minute (key quota) and 3 simultaneous requests (parallel cap). Capture cooldown 429s passively from normal traffic. Record the status (429 or 503), body text, and the `Retry-After` format (integer, float, or HTTP-date).	Gives the real bodies to match in T1-5. A key-quota Retry-After of about 60 s means LiteLLM's v3 fixed-window limiter; a fractional value means v1 (v3 limiter, v1 limiter).
P8	Window semantics	Log `remaining-requests` and `remaining-max_parallel_requests` on every response for 3 minutes. Note when `remaining` resets relative to the first request in the window.	A reset anchored to the first request means a fixed window → resync the local bucket from headers (T1-5).

#	Probe	How to run it	How to read it → what it decides
P9	Prefix-cache latency effect	Build a fixed ~1,000-token system prompt S. Send (a) S + q1 twice, (b) S + q2, (c) a cold prompt S' + q1 of the same length, (d) the same sequence interleaved with a classify prompt on the same model.	If warm prompts are at least 25% faster → enforce a shared prefix (T2-7). No difference → keep the layout anyway (it costs nothing) but don't claim a gain.
P10	Concurrency cost	Send 6 equal-size requests one at a time, then the same 6 as 3 concurrent pairs. Compare `service_ms`.	If per-call service time roughly doubles at 2 in flight → adaptive limit $L \in \{1, 2\}$ (T2-4).
P11	Token counting and truncation	Try `POST /tokenize`, `/utils/token_counter` and `/v1/responses/input_tokens` (LiteLLM count_tokens). Then the canary test: put "the code word is ORANGE" first and "end your answer with 7319" last, and pad the prompt to 1,500 / 1,800 / 2,000 / 2,200 / 2,600 tokens, in French and in Arabic.	Gives the real cutoff, which end gets cut, and characters per token for each script. Sets the budget in T1-4.
P12	Throughput and usage fields	For each model, compute `usage.completion_tokens / service_s`. Record classify's `completion_tokens` and check whether usage fields are present at all.	Classify above 20 output tokens → T1-2 is confirmed.
P13	Cold start	Leave the model idle for 6 minutes or more, then time one request against a warm one. Ollama unloads idle models after 5 minutes by default (Ollama prompt caching ⚠).	Noticeably slower cold request → keep-warm ping (T2-7).
P14	Server-side retries and timeouts	Send the `x-litellm-num-retries: 0` and `x-litellm-timeout: 60` headers and see whether the proxy accepts them (LiteLLM routing, timeouts).	Accepted, and allowed by the rules → T2-8.
P15	Compact accuracy mini-set	30 bank items (solve, value-at, tangent) on the compact model, returning a fixed `answer` field; check each with sympy. If P1 found new models, also run a 12-prompt smoke test per model (FR/EN/MSA/Darija/Arabizi).	Compact accuracy ≥ 60% and sympy parses ≥ 50% of items → compact-first cascade (T2-5).

#	Probe	How to run it	How to read it → what it decides
			New models → full decision test (T2-6).

Ten upgrades come first, eleven wait on Monday

ID	Improvement (upgrade of an existing component)	Metrics	Effort	Monday trigger
T1-1	Sampling profiles per model and task; fix the fp mapping; loop guard regenerates once	RS, RL	0.5 pd	P2, P3 set the values
T1-2	Terser classify, per-task `max_tokens`, `finish_reason` handling	CH, OS	0.5 pd	P12
T1-3	Confidence-gated rule shortcut plus a budget-targeted escalation threshold	**OS**	1.5 pd	Always; calibrate on Monday logs
T1-4	Real token counting, truncation canary, byte-stable prompt layout, deterministic small-to-big packing	**CH**	1 pd	P11, P9
T1-5	429 taxonomy, model-scoped cooldown pause, header resync, retry budget, no self-inflicted cooldowns	**RS**, CH	1 pd	P7, P8
T1-6	Cache key hygiene, canonical L1.5 key, tier tags, repeated-request proof	**RR**	1-1.5 pd	Always
T1-7	Sympy fact sheet, multi-claim checker, equivalence-based leak guard, parser hardening	**RL**	2-2.5 pd	Always
T1-8	Math-structure detector and fuzzy glossary in retrieval	**RL**	0.5 pd	Always
T1-9	Term lock, wrong-dialect markers, script intruders for Arabic output	RL	0.5-1 pd	Always
T1-10	Judge-facing metrics: OTel-named JSONL fields and a one-page dashboard	all	0.5 pd	Always
T2-1	Strict `json_schema` for classify and verdicts (reasoning field first)	CH, RS	0.5 pd	P5 enum obeyed
T2-2	Logprob margin as the routing confidence signal	OS	0.5 pd	P4 non-null
T2-3	Backend-specific samplers (DRY on llama.cpp; repeat_penalty 1.05 / min_p on Ollama)	RS, RL	0.25 pd	P3 shows passthrough
T2-4	Adaptive concurrency $L \in \{1, 2\}$ with a latency guard	RS, CH	0.5-1 pd	P10 shows about 2× slowdown

ID	Improvement (upgrade of an existing component)	Metrics	Effort	Monday trigger
T2-5	Compact-first cascade that escalates when sympy disagrees	**OS**	1 pd	P15 ⩾ 60%
T2-6	Per-language model routing / swap to Qwen3-4B or Gemma-3-4B	RL, OS	0.5-1 pd	P1 lists new models
T2-7	Shared cross-task prefix and keep-warm ping	CH	0.25 pd	P9 ⩾ 25%, P13
T2-8	Turn off hidden proxy retries	RS	0.1 pd	P14 accepted
T2-9	Use the LiteLLM cache namespace / `no-store` for personalized responses	RR	0.25 pd	P6 header present
T2-10	Deadline-aware admission with a downgrade ladder	RS, CH	1 pd	Real interactive p95 > 25 s
T2-11	Hint self-consistency checked by sympy, not votes	RL	0.25 pd	Quota headroom at demo load
T3-1… T3-10	Week-two items: buggy-rule diagnosis, tutor state machine, warmed concept bank, doc2query, canonical-intent cache key, fault-injection server, fair queuing, staged progress UX, CAMeL DID / TArC variants, variant generator	mixed	0.5-3 pd each	See each item

Tier 1 adds up to about 9-10 pd, which is roughly one working week for two people. If time runs short, work through it in this order: T1-1, T1-5, T1-2, T1-4, T1-3, T1-6, T1-10, T1-7, T1-8, T1-9. That order puts the items that protect the resilience and constraint-handling scores first.

Tier 1: upgrades worth doing whatever Monday shows

T1-1. Replace the global penalty with per-model sampling profiles

What. Drop the engine-wide `frequency_penalty: 0.3`. Replace it with a profile table keyed by (model, task) that sets temperature, top_p, max_tokens, stop and an optional `extra_body`. Also upgrade the loop guard so it regenerates once instead of only truncating.

Why. Two mirrored LiteLLM versions send OpenAI `frequency_penalty` unchanged to Ollama as `repeat_penalty`, drop `presence_penalty`, and forward unknown kwargs into Ollama `options` (older commit, newer branch) ⚠ (version 1.103 not inspected). Even with true OpenAI semantics, frequency penalty is a weak loop fix. A 2026 study on Qwen2.5-1.5B,

Llama-3.2-3B and Qwen2.5-7B measured the loop metric at 0.124 with no intervention, 0.097 with frequency penalty, 0.083 with repetition penalty and 0.065 with DRY (Weidmann et al. ⚠). Temperature cuts both ways. Greedy decoding drives self-reinforcing sentence loops (Xu et al., NeurIPS 2022), while higher temperature increases wrong-language output (Marchisio et al., EMNLP 2024). That points to a low but nonzero temperature with a truncated tail. Qwen's own defaults are repetition_penalty 1.05, top_p 0.8, top_k 20 (Qwen HF commit).

Impact. RS (fewer loop salvages) and RL (fewer stray-language tokens). If the mapping bug is real, the loop rate could drop a lot.

Steps.

1. Create `sampling_profiles.yaml`. Key it by `model_id`, then by task (`classify`, `verdict`, `explain_body`, `hint`). Fields: `temperature, top_p, max_tokens, stop, extra_body`. Load it at startup and put `profile_hash` in every JSONL record.
2. Defaults: classify and verdict at temperature 0-0.1; explain body and hints at temperature 0.3-0.4 with top_p 0.8-0.9. Leave out `frequency_penalty` and `presence_penalty` everywhere.
3. Set the penalty from the P2 result. If fp is mapped to repeat_penalty, use `frequency_penalty: 1.05` on Ollama-backed prose tasks only (it becomes repeat_penalty 1.05), or `extra_body: {"repeat_penalty":1.05,"repeat_last_n":128}` if P3 shows passthrough. If OpenAI semantics apply, use fp ⩽ 0.3 or none.
4. Loop guard: also detect repeated sentences (normalized-word sentence hash seen twice or more), not only 3-grams. When a loop is detected and `remaining_requests` > the reserve, regenerate once with a new seed and temperature + 0.2. Otherwise keep the existing salvage path.
5. Re-run 200 explanations per profile. Report the loop rate and the sympy-correctness rate side by side, because repeat_penalty above 1 also penalizes the "x", "=" and digits that math has to repeat.

Effort / risk / trigger. 0.5 pd. Risk: a penalty above 1.05 corrupts equations, so watch the correctness column. Trigger: always build the profile table; P2 and P3 decide the values.

T1-2. Put classify on a token diet and cap every task

What. Make classify emit a minimal JSON object with short keys and single-character codes, and set hard `max_tokens` per task with clean handling of `finish_reason == "length"`.

Why. Output generation is almost always the slowest step, and halving output tokens can roughly halve latency (OpenAI latency guide). CPU generation runs at only about 3-10 tokens/s for 3-9B models (itsfoss CPU test ⚠, mgroeber benchmarks ⚠, both informal). At those speeds, a 2.6-5 s classify corresponds to about 25-45 output tokens, so cutting it to about

10 tokens should bring it to roughly 1-2 s (inference). Caps also protect against Caddy's 90 s timeout: 300 tokens at 3-4 tokens/s on an 8B model is 75-100 s.

Impact. CH and latency. It also protects RS, because shorter calls mean fewer timeouts and less time spent holding a concurrency slot.

Steps.

1. Change the classify output to something like `{"i":"H","l":"fr","c":3,"x":1}`: intent code, language code, complexity 1-5, confusion flag. No reasoning field. Use `max_tokens: 24`, `temperature: 0`, and `stop: ["}"]`, then re-append `}` before parsing.
2. Map codes to enums in Python and keep the existing JSON repair ladder.
3. Per-task caps: compact hints 60-80 tokens; mid-size explain body 220-300 tokens, plus a prompt instruction of "at most 5 short steps".
4. On `finish_reason == "length"`, trim back to the last complete sentence or step, and log `truncated_output = true`.
5. For worked solutions, ask for structure (`steps: ["…", "…"]` as short equations) and render the connective text in code from FR/EN/AR templates, which costs no tokens (Chain of Draft ⚠, abstract only). Use this only for internal and verification text, not learner-facing prose, until it has been A/B tested on the 8B model.

Effort / risk / trigger. 0.5 pd. Risk: terse codes can confuse the 3B model, so compare classify agreement on 50 logged turns before and after. Trigger: P12 shows classify above 20 completion tokens, which is very likely.

T1-3. Let rules skip classify, and tie escalation to the quota budget

What. The engine already runs rules plus one compact classify call. The upgrade has three parts:

1. When a rule class has proven agreement with the classifier, skip the classify call entirely.
2. Replace the fixed "complexity ⩾ X → mid-size" rule with a threshold calibrated to a target mid-size share. The threshold tightens as quota drains or mid-size latency rises.
3. Publish the split.

Why. A 2026 routing survey recommends a layered design: a cheap pre-router, a post-generation check and an escalation rule. It warns that routers which spend extra calls add latency, and that hard queries sent to weak models cost more than the reverse (Moslem & Kelleher, TMLR survey). RouteLLM calibrates its threshold to a target "strong-model %" on the team's own queries (RouteLLM), and Hybrid LLM keeps the quality target tunable at test time, cutting large-model calls by up to 40% with no quality drop (Ding et al., ICLR 2024). Do not merge free-form generation into the compact classify call. Small models lose accuracy on multi-instruction prompts, especially for free-form generation (MTI Bench).

Impact. OS, directly. Each skipped classify saves one request and 2.6-5 s. If rules cover about half of turns confidently, requests per turn fall from about 2.08 to about 1.6 (inference; depends on the traffic mix).

Steps.

1. Log at least 100 turns (load tests plus Monday) with the rule features (length, operators or LaTeX, "why" keywords in FR/EN/AR/Darija/Arabizi such as pourquoi, 3 ,علاشlech, script, digits inside words) next to the classify output.
2. For each rule class, compute agreement with classify. Enable the shortcut only where agreement is ⩾ 95%. Log `route_source = rule | llm`.
3. Keep complexity as an integer from 1 to 5. From the logged distribution, choose the cutoff that gives a target mid-size share, for example 25%.
4. At runtime, raise the cutoff by one step when local bucket tokens < 5, when the mid-size breaker is half-open, or when mid-size p95 > 20 s. Lower it again after 2 clean minutes. Expose the current cutoff on the metrics endpoint.
5. Optional: a compact classify may also return a short extractive field, such as `"ans":"x=3"`, for sympy grading. Short extractive fields are the multi-task case that small models handle best.
6. Report the split per intent, plus AutoMix's incremental benefit per cost, IBC = (quality_mixed − quality_compact) / (mid-size calls_mixed − mid-size calls_compact) (AutoMix), so judges see the trade-off quantified.

Effort / risk / trigger. 1-1.5 pd. Risk: code-switched messages break the rules, so keep the LLM classify as the fallback for every uncovered case. Trigger: always; set the calibration from Monday logs.

T1-4. Count real tokens, find the truncation edge, keep prefixes byte-stable

What. Count tokens with the real tokenizer before every call. Use a measured budget with a 10% margin, put the question last, keep a byte-identical system prefix, and make context packing deterministic.

Why. Truncation is silent, so a 2-characters-per-token heuristic either wastes budget or, for Arabic-heavy input, may overflow. llama.cpp exposes `POST /tokenize` (llama.cpp REST overview ⚠, mirror), and the LiteLLM proxy documents count routes that fall back to local tiktoken counting (LiteLLM count_tokens). llama.cpp and Ollama reuse a slot's KV cache only for the matching prefix and recompute from the first differing token (llama.cpp #13606). One Ollama example showed a 17.7× prefill speedup (Leanpub ⚠, GPU example). Sentence-window retrieval (index small, show the surrounding window) slightly beat plain chunking and recovered answers buried mid-chunk (LlamaIndex demo). Models use the start and end of the context better than the middle (Lost in the Middle summary ⚠).

Impact. CH (no silent truncation, a provable budget) and latency (shorter prefill on CPU, plus prefix reuse).

Steps.

1. `token_counter.py`: use the P11 endpoint if one exists. Otherwise load each hosted model's Hugging Face tokenizer with `tokenizers` on CPU. If that fails too, fall back to the per-script characters-per-token ratios measured in P11.
2. `BUDGET = measured_cutoff × 0.9` (about 1,800 if the cutoff is 2,048). Assert the budget before every send, and log `prompt_tokens_est` and `truncation_suspected`.
3. Fixed layout per task: `messages[0]` is a module-level constant with no f-strings, dates, learner names or IDs. Language- and level-specific static instructions come next. Retrieved chunks follow in a deterministic order (score, then chunk_id). A code-built state line such as `topic=produit; last_error=sign; attempts=2` comes after that, and the learner question comes last. Add a unit test asserting that `messages[0]` is byte-identical across calls for each task.
4. Packing: if $score_2 < 0.5 \times score_1$, send only chunk 1, in full. Otherwise send chunk 1 in full plus a ±2-sentence window around chunk 2's best-matching sentence (scored with the same BM25 at sentence level). If the current extractive compression makes an LLM call, keep that call only as a fallback when this deterministic window still doesn't fit.
5. Prepend a 10-15-token deterministic header to each chunk, for example `[Dérivées › Règle du produit]`.
6. Add the P11 canary as a nightly regression test.

Effort / risk / trigger. 1 pd. Risk: if P11 shows the proxy cuts the front, the system prompt is what gets lost. In that case shorten it and repeat the single critical rule at the end of the user turn. Trigger: P11 sets the budget, and P9 decides whether to advertise the prefix gain.

T1-5. Tell the three kinds of 429 apart and stop self-inflicted cooldowns

What. Upgrade the existing Retry-After handling and circuit breaker in five ways:

1. Classify every refusal as `key_quota`, `deployment_cooldown` or `parallel_cap`.
2. On a cooldown, pause the whole model for the stated period.
3. Resync the local bucket from the rate-limit headers after every response.
4. Add a retry budget.
5. Stop sending requests that themselves trigger cooldowns.

Why. LiteLLM's default deployment cooldown is 5 s and applies per deployment, so it is shared by every key on the proxy. It fires immediately on a 429, on 401/404/408, or when more than 50% of a minute's requests fail (LiteLLM routing, cooldown_handlers source). The observed `Retry-After: 5` therefore means "the backend is cooling down", not "your key is out of quota".

The v3 key limiter is a fixed window anchored to the first request in the window and returns `retry-after = 60`, the whole window (v3 limiter). Pacing ahead of the limit is therefore far cheaper than reacting to 429s. Google SRE caps retries at 3 attempts per request and keeps them below 10% of a client's traffic, and says not to retry when the backend is broadly overloaded (SRE book).

Impact. RS and CH. Judges see refusals per key, so breaking them down by cause and showing how many were avoided is directly scorable.

Steps.

1. `classify_refusal(status, body, headers)`: body matches `Rate limit exceeded for api_key` → `key_quota`. Body contains `No deployments available` or `all_deployments_in_cooldown` (status 429 or 503) → `deployment_cooldown`. Body matches the parallel-request limit → `parallel_cap`. Fit the patterns to the real P7 bodies.
2. Parse Retry-After as an integer, a float (v1 sends values like `"37.42"`), or an HTTP-date.
3. `deployment_cooldown` → open a model-scoped gate until `Retry-After + U(0, 1.5 s)` and release queued requests **one at a time**, so waiters don't stampede at t + 5 s. This sits apart from the existing breaker (3 failures in 60 s). `key_quota` → no calls to any model until the estimated window reset, interactive traffic first.
4. After every response, set `bucket.tokens = min(bucket.tokens, remaining_requests)`. When `remaining ⩽ 3`, admit only interactive traffic and pause background work.
5. Retry budget: retries ⩽ 10% of requests over a sliding 2-minute window and at most 3 attempts per request. When the budget is exhausted, return the degraded answer.
6. Avoid self-inflicted cooldowns: validate model names against `/v1/models` at startup, never retry 401 or 404, and keep client timeouts below Caddy's 90 s (for example 75 s total, with `x-litellm-timeout` slightly lower if P14 shows the header is accepted).
7. Add to JSONL: `refusal_kind`, `retry_after_s`, `ratelimit_remaining_requests`, `ratelimit_remaining_parallel`, `gate_wait_ms`, and an `avoided_429` counter (incremented each time the gate waited because `remaining` was low).

Effort / risk / trigger. 1 pd. Risk: misreading a cooldown as a key-quota 429 over-throttles for 60 s, so match on body text, not only the status code. Trigger: P7 and P8 provide the exact shapes; the logic itself is always needed.

T1-6. Make the cache key honest, canonical, and visible

What. Make five changes to the existing exact cache, negative cache and coalescing:

1. Add `kb_version` and `guard_version` to the key.
2. Chain follow-up turns to the previous turn's key.

3. Add an L1.5 canonical exact key: still an exact match, but on normalized input.
4. Re-run the cheap guards when serving a hit.
5. Tag every response with the cache tier that served it.

Why. Embedding-free canonicalization into a structured signature raised the hit rate from 28% (raw text) to 82% with zero false hits on 1,395 controlled queries (LLMSigCache). Query preprocessing alone added 46-48% token savings in SCALM's ablation (SCALM). Without context chaining, follow-up questions false-hit badly: 54 of 100 for GPTCache versus 3 of 100 for MeanCache (MeanCache). Practitioners advise version-tagged invalidation and tracking a "corrected cache rate" (Tian Pan ⚠, practitioner). EdNova's 28-55% exact hit rate is already at the top of the 15-30% typical for exact-match layers, so expect a gain of a few points, not a doubling.

Impact. RR, plus correctness (a stricter guard no longer serves answers that only passed the old one).

Steps.

1. Key = sha256(task, model_class, `canon(prompt)`, lang, level, prompt_version, `kb_version` = hash of glossary + chunks, `guard_version`, chunk_ids, `prev_turn_key` for follow-ups).
2. `canon()`: NFKC normalization; casefold Latin; the existing Arabic normalizer (tatweel, diacritics, alef, ta marbuta, alef maqsura); Arabic-Indic digits ٩-٠ → 0-9; math spans `x²→x^2`, `×·→*`, `÷→/`, and digit-comma-digit → `.`; whitespace stripped inside math spans; optionally sympy `srepr` for spans that parse; a tiny per-language list of filler words removed (svp, stp, please, 3afak).
3. Hard rules: never treat a digit inside a word as a number (Arabizi 3lech, 7aja); never strip negations (ne…pas, موش, mouch); keep the task verb, since simplifier ≠ développer.
4. On every non-L1 hit, re-run the deterministic guards (script, leak, math claims) before serving.
5. Add `cache_tier ∈ {L0 coalesced, L1, L1.5, miss}`, `tokens_saved` and `quota_minutes_saved = avoided / 20` to JSONL.
6. Add a `replay_identical.py` test: N identical requests, both in parallel and in sequence, should show about 100% L0/L1 and zero upstream calls after the first.

Effort / risk / trigger. 1-1.5 pd. Risk: an over-aggressive `canon()` merges different exercises, so unit-test pairs like `2x+3=7` vs `2x+3=9`. Trigger: always add the hygiene. Ship L1.5 if grouping a real log by L1.5 key collapses at least 3-5% of requests into keys that already exist.

T1-7. Ground every formula in a sympy fact sheet

What. The engine already does sympy grading and derivative-claim checks. Upgrade to "compute, then verbalize":

1. Before the LLM call, sympy builds numbered facts: f, f', simplified f', f'(a), f(a), the tangent at a, critical points and the sign table.
2. The prompt says "use only these formulas".
3. A claim extractor checks every formula and number in the output against the facts.
4. The leak guard compares by sympy equivalence, not by string.

Why.

- Program-aided reasoning, where code computes and the LLM narrates, beat chain-of-thought PaLM-540B by 15 points on GSM8K (PAL).
- Grounding tutor feedback in verifier output reduced hallucinations (Daheim et al. ⚠, preprint).
- Unguarded GPT-4 answered the study's problems correctly only about 51% of the time, and the guarded "GPT Tutor" was fed teacher solutions and common mistakes (Bastani et al., PNAS 2025 preprint).
- Khan Academy reports that models often mark wrong answers right and vice versa (Khan Academy blog).
- `parse_expr` uses `eval` and must not see unsanitized input (SymPy parsing docs).

This item targets EdNova's measured failures: 1/91 wrong math, 2 ungrounded formulas and 10 invented examples.

Impact. RL and grounding. Invented and ungrounded content becomes detectable by construction, with no extra LLM calls.

Steps.

1. `build_facts(exercise) -> list[Fact]` renders lines such as `F1: f(x) = x^3 - 3x` and `F2: f'(x) = 3x^2 - 3`. Insert them into the explain prompt with "Use ONLY F1… Fn. Do not introduce other functions or numbers."
2. `extract_claims(text, lang)`: regexes for FR and EN covering `f'(x)=E`, la dérivée de f est E, `f'(a)=v`, `f(a)=v`, the tangent `y=mx+p` (tangente au point d'abscisse a), croissante/décroissante sur I, and extremum en x=a. Normalize unicode first (²→**2, −→-, ×·→*, digit-comma-digit → decimal point, ln→log).
3. Check each claim with sympy inside a thread timeout of about 2 s: `simplify(a-b)==0`, falling back to numeric sampling at random in-domain points with tolerance 1e-9. Monotonicity uses `solve_univariate_inequality(diff(f) >= 0, x,`

`relational=False)` containing I. Verify that this helper behaves as expected in the installed sympy version; it was not tested during research.

4. Any `name(x) = E` that isn't equivalent to a fact → `ungrounded`. Any expression equivalent to the asked answer (f', f'(a), the tangent) in a hint → `leak`.
5. On failure, regenerate once at temperature 0.1. On a second failure, use the template-only explanation.
6. Parser hardening: `transformations = standard_transformations + (implicit_multiplication_application, convert_xor, rationalize)`. Pre-tokenize `ln`, `exp`, `sin`, `cos` and `sqrt` and don't use `split_symbols`. Whitelist characters and function names before calling `parse_expr(..., global_dict=SAFE)`. If LaTeX input is ever accepted, use `parse_latex(s, backend="lark")`, which rejects ill-formed input instead of silently half-parsing it.
7. Log `claims_extracted`, `claims_checked`, `claims_failed` and `leaks_caught` per response.

Effort / risk / trigger. 2-2.5 pd. Risk: limited regex recall (for example "three x squared" written out in words), so report claims checked per explanation instead of claiming full coverage. Trigger: always.

T1-8. Teach retrieval to see math structure

What. Before BM25 runs, a regex detector recognizes products, quotients, compositions, powers and constants in the query and injects concept tokens. A fuzzy fallback maps misspelled tokens to glossary keys. The injected tokens count as covered terms in the coverage gate.

Why. Switching BM25 variants barely changes results (39.5-39.9 nDCG@10 on BEIR); tokenization drives most of the difference (BM25S paper). Tangent-L turns formulas into structural tokens and mixes them with text at a math weight of α = 0.25, the best answer-retrieval run at ARQMath-2 (Ng et al.). The known miss, "Pourquoi la dérivée de x(x+1) n'est pas 1×1 ?", is exactly a product-structure problem.

Impact. RL. It should fix the x(x+1) miss and keep formula-heavy, word-light questions such as "(x²+1)' ?" from being rejected as off-topic.

Steps.

1. `detect_structure(q)` runs on the normalized query. Product: `\)\(`, `[a-z0-9]\(`, `\)[a-z]`, or `*` between factors that contain x → add `produit regle_produit`. Quotient: `/` with x in the denominator → `quotient`. Composition: `f(g(x))`, `(…x…)^n`, `sqrt(…x…)`, `sin(kx)` → `composee chaine`. Power: `x^n` → `puissance`. Patterns like "1 × 1" or `u'v'` → `erreur_courante`. Require a variable inside or next to the parentheses.

2. Add the hand-written concept tags (from the 60 glossary concepts) to each chunk's indexed text.
3. Fuzzy glossary: for unknown tokens of 4+ characters, `rapidfuzz.process.extractOne(tok, glossary_keys, scorer=fuzz.ratio, score_cutoff=85)` (standard API, not checked against the docs during research ⚠). Log tokens that stay unmapped so they can be added to the glossary by hand.
4. Adoption bar: the x(x+1) case moves to rank 1 and no other hit@1 changes.

Effort / risk / trigger. 0.5 pd. Risk: false positives on parentheses in prose (mitigated by the variable requirement). Trigger: always.

T1-9. Lock math terms and dialect markers in Arabic output

What. Three post-generation checks on Arabic-script output:

1. A term lock that replaces wrong MSA terms with the Tunisian school terms.
2. A list of non-Tunisian dialect markers that triggers a regeneration or the template.
3. A wider script guard that also catches Indonesian/Malay words and Chinese characters.

Why. "Most LLMs struggle to recognize and respond properly in Tunisian Arabic" (TounsiBench, EMNLP 2025). LLMs default to MSA and understand dialect better than they generate it (AraDiCE), and few-shot examples overcome much of that reluctance (AL-QASIDA). Five target-language examples raised one model's cross-lingual line-level pass rate from 1.1% to 95.0% (Marchisio et al.). Tunisian students learn mathematics in Arabic up to the 9th year of basic school and in French from lycée onward (IREM Grenoble), so collège learners know the Arabic school terms. The observed failures (ديال, الوظيفة for "function", an Indonesian token) match these documented patterns.

Impact. RL (correct terminology for the learner's level, no Moroccan drift). This matters most because Arabic output comes from the mid-size model.

Steps.

1. `term_lock.json` maps each concept to `{fr, ar_school, do_not_use[]}`, for example `fonction: دالّة, do_not_use: [وظيفة]`, `dérivée: مشتقّة`, `tangente: مماس`, `vecteur: شعاع`. ⚠ This list comes from general knowledge, not a source; verify it against the CNP 8e/9e textbooks. Apply a regex replacement after generation, and add the list to the system prompt as required terminology.
2. Wrong-dialect markers (⚠ general knowledge; native speakers must validate): ديال، بزاف، دابا، كاين، مزيان، بغيت، عايز، دلوقتي، إزاي. Any hit → one regeneration with Tunisian examples, then the curated template.
3. Script guard: flag the CJK range on Qwen outputs, and keep a Latin-in-Arabic intruder list (maka, jadi, adalah) that excludes variable names and math symbols.

4. Use one prompt language per call: system prompt and instructions in the output language, with only the JSON keys in English.
5. Add regression unit tests for the three observed failures.

Effort / risk / trigger. 0.5-1 pd. Risk: over-substitution changes meaning, so keep the table small and limited to math context. Trigger: always.

T1-10. Show the judges the numbers they score

What. Rename and extend the JSONL fields to follow the OpenTelemetry GenAI conventions, and generate one static HTML dashboard from the JSONL.

Why. The OTel GenAI semantic conventions define `gen_ai.client.operation.duration` and `gen_ai.client.token.usage` with standard attributes. They are still in "Development" status (OTel GenAI metrics). AWS defines goodput (answers delivered on time and without error) as the number to watch under overload, and recommends recording queue time explicitly (AWS Builders' Library).

Impact. Every metric: it makes OS, CH, RS and RR visible.

Steps.

1. Fields: `gen_ai.operation.name`, `gen_ai.request.model`, `gen_ai.response.model`, `gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens`, `error.type`, `queue_ms`, `service_ms`, `total_ms`, `attempt_no`, `priority_class`, `learner_id_hash`, `route_source`, `model_tier`, `cache_tier`, `refusal_kind`, `outcome ∈ {ok, degraded, shed, cache_hit, coalesced}`, `limit_L`, `prompt_tokens_est`.
2. Dashboard panels: goodput vs offered load; interactive p50/p95 split into queue and service time; attempts per logical request; 429s per 100 requests by kind and avoided 429s; cache savings by tier; compact/mid-size split by intent; degraded share with reasons; quota utilization against 20/min.

Effort / risk / trigger. 0.5 pd. Trigger: always.

Tier 2: build only if Monday's probes say so

T2-1. Strict `json_schema` for classify and verdicts (trigger: P5 output obeys the enum)

Real schema-constrained decoding exists at all three layers:

- llama.cpp converts `response_format: json_schema` into a grammar, but silently ignores schema keywords it doesn't support (Liquid AI on llama.cpp).
- Ollama accepts a schema in `format` (Ollama blog).
- LiteLLM's newer `ollama_chat` code maps `json_schema` to that field (mirror ⚠).

Field order matters. Forcing reasoning into JSON dropped LLaMA-3-8B-Instruct from 70.1% to 28.0% on one task, mostly because models wrote "answer" before "reason". Classification-style outputs did as well or better under JSON mode (Let Me Speak Freely?, EMNLP 2024). With good schemas, constrained decoding improved accuracy by up to 4% (JSONSchemaBench, ICML 2025).

Steps:

1. Use flat schemas with `additionalProperties: false`, `enum` for every code, and `maxLength` on strings. For verdicts, the order is `reasoning` (maxLength 400) → `verdict` (enum) → `confidence` (enum).
2. Fallback ladder: json_schema → json_object with the schema pasted into the prompt → local Pydantic validation → one repair call → safe default.
3. Never wrap explanations in JSON.
4. Re-test mistral-7b and qwen2.5-coder-7b with a schema before dropping them. Their "JSON failure" is probably proxy configuration, not the models.

Impact: CH and RS (fewer repair calls). 0.5 pd. Risk: grammar compilation latency on CPU, so keep schemas small.

T2-2. Logprob margin as the routing signal (trigger: P4 returns non-null logprobs)

Ollama 0.12.11 added logprobs on its OpenAI-compatible API (release notes ⚠, secondary), but LiteLLM's provider table shows no Ollama logprob support (LiteLLM input params), so only the probe can tell. For short outputs, quantiles of per-token probability (for example the minimum) are better deferral signals than summed logprobs (Gupta et al., ICLR 2024).

Steps:

1. Make the complexity code a single known token.
2. Request `top_logprobs: 5`.
3. Compute top-1 probability, the margin, and the expected complexity $\Sigma\, p_i \cdot i$.
4. Treat a margin below 0.2 as "uncertain" and route conservatively, i.e. to mid-size if there is budget, otherwise compact plus verification.
5. Handle `logprobs is None`, because `drop_params` can strip the field silently.

Impact: OS (fewer wrong escalations), at zero extra requests. 0.5 pd.

T2-3. Backend-specific anti-repetition samplers (trigger: P3 shows the extra body is applied)

On a **llama.cpp** backend, DRY cut verbatim loops by 47% on 1.5-7B models with no measured capability loss, using $\lambda = 0.8$, $\beta = 1.75$, allowed_length = 2 and a 1,024-token window (Weidmann et al. ⚠, Aug 2026, not peer-reviewed). llama.cpp exposes the `dry_*` parameters (llama.cpp

sampling ⚠, mirror). Pass `extra_body={"dry_multiplier":0.8,"dry_base":1.75,"dry_allowed_length":2,"dry_penalty_last_n":1024}` on prose tasks only.

On an **Ollama** backend, DRY isn't available (smcleod guide ⚠, secondary). Use `repeat_penalty: 1.05`, `repeat_last_n: 128-256`, `min_p: 0.05` and `top_k: 20` through the extra body. Min-p's gains are disputed by a 2026 re-analysis (critique listing), so measure stray-token rate per 1,000 outputs with and without it.

Never use `no_repeat_ngram` (see the skip list). Impact: RS and RL. 0.25 pd.

T2-4. Adaptive concurrency with a latency guard (trigger: P10 shows about 2× slowdown at 2 in flight, or cooldown 429s caused by other teams)

When the ceiling is 2, adaptivity is useful for backing off, not for raising throughput. If a second concurrent call just doubles each call's latency on CPU, a limit of 1 gives the same throughput with half the per-request latency. Use Inspect AI's controller pattern: multiply by 0.8 on a 429, at most one cut per cooldown window, and treat 5xx errors as "pause growth, don't shrink" (Inspect concurrency). Add a Gradient-style guard that shrinks when latency exceeds 2× the observed minimum (Netflix GradientLimit).

Steps:

1. Merge the semaphore and the bucket into one permit controller with $L \in \{1, 2\}$.
2. On a `deployment_cooldown`, set $L \leftarrow 1$, debounced to one cut per Retry-After window.
3. Set $L \leftarrow 1$ when the EWMA of `service_ms` exceeds $2 \times$ the minimum observed.
4. Set $L \leftarrow 2$ after 10 clean successes.
5. Log `limit_L`.

Impact: RS and CH, and better interactive p95. 0.5-1 pd. Note: no published evidence exists for AIMD at a ceiling this small, so justify it with your own JSONL.

T2-5. Compact-first cascade with sympy as the verifier (trigger: P15 compact accuracy ⩾ 60% and sympy parses ⩾ 50% of items)

Cascades gated by verification or consistency cut strong-model cost by 50-60% at comparable quality (AutoMix, Mixture-of-Thoughts cascade). AutoMix's verifier samples an LLM 8 times per query, which is impossible at 20 requests per minute. Sympy replaces that verifier at zero request cost.

Steps:

1. For solve/value/tangent items, the compact model returns `{"answer": "..."}`.
2. Sympy compares the answer to its own solution: equality, or substitution back into the equation.

3. Agreement → accept the compact answer and verbalize it with the T1-7 facts.
4. Mismatch or unparseable → escalate to mid-size. If the breaker is open, fall back to the template.
5. Log the accept rate and IBC.

Impact: OS (mid-size calls only where needed; 6-30 s drops to compact latency on accepted items). 1 pd. Risk: false accepts if parsing is sloppy, so require exact symbolic equality. Conceptual "why" questions still go through the router.

T2-6. Route by language, or swap models (trigger: P1 lists new models)

On the 3LM Arabic STEM benchmark, Qwen3-8B, Qwen2.5-7B and Qwen3-4B lead, Gemma-3-4B sits in the middle, and Llama-3.1-8B is last (synthetic math MCQ 53.5 vs 81.2 for Qwen3-8B, base models) (3LM). Llama 3.2 does not officially support Arabic (Llama 3.2 card). Self-reported IFEval scores are 90.2 for Gemma-3-4B-IT (Gemma 3 card) and 83.4 for Qwen3-4B-Instruct-2507 (Qwen3 README mirror), against 77.4 for Llama-3.2-3B. The evidence conflicts: AL-QASIDA found Llama models best at keeping the requested dialect (AL-QASIDA), even though Qwen leads on Arabic math.

So don't choose from leaderboards. Run 60 prompts (12 per language variant) on each model and score sympy correctness, dialect/script markers, tone rules, loop rate, JSON validity and p50/p95 latency. Then set defaults per language, for example Qwen for MSA math, Llama for FR/EN, and whichever wins the Darija markers for Darija spans. Ask the organizers to host ESPRIT-Derja-Qwen3-8B-v2 only after confirming its license and base model, which were not stated (ESPRIT-Group HF). For Qwen3 on llama.cpp or vLLM, the official advice is `presence_penalty` 0-2 against endless repetition, with a warning that high values cause language mixing (Qwen3 card mirror). LiteLLM's Ollama path drops this parameter.

Impact: RL and OS. 0.5-1 pd plus about 180-300 requests at background priority.

T2-7. Shared prefix across tasks and a keep-warm ping (trigger: P9 ⩾ 25% faster warm; P13 cold start noticeably slower)

If classify, hint and grading run on the same compact model, open all three with one long shared block (role, language rules, notation rules) and put the task-specific instructions after it, so they share a cached prefix in a slot. With 2 slots shared across tasks and tenants, caches get evicted (llama.cpp #13606), so measure before claiming any gain. If P13 shows a cold-start penalty, send a 1-token request every 4 minutes during demos, about 15 requests per hour. 0.25 pd. Impact: CH and latency only. Neither change saves quota.

T2-8. Turn off hidden proxy retries (trigger: P14 header accepted and allowed by the platform rules)

The LiteLLM router retries rate-limit errors with backoff, and the `x-litellm-num-retries` header takes precedence over other settings (LiteLLM routing). Send `x-litellm-num-retries: 0` so the server's retries don't eat into EdNova's own retry budget. 0.1 pd. Impact: RS (accurate attempt accounting).

T2-9. Cooperate with the proxy cache (trigger: P6 shows `x-litellm-cache-key`)

Per-request controls such as `cache: {"namespace": ..., "no-store": true, "ttl": ...}` go in the request body (LiteLLM cache controls). Send `no-store` for personalized responses, use a namespace per `prompt_version`, and log whether proxy hits decrement `remaining-requests`. Keep EdNova's own cache as the source of truth. 0.25 pd.

T2-10. Deadline-aware admission with a downgrade ladder (trigger: real interactive p95 > 25 s; load tests already showed 34-60 s under overload)

AWS recommends an upper bound on how long a request can sit in a queue, propagating deadlines, and skipping work the client has already abandoned (AWS Builders' Library). Nielsen's 10 s attention limit applies (NN/g).

Steps:

1. Interactive deadline 25 s; background 120 s.
2. At grant time, if `now + ETA_queue + p50_service > deadline`, downgrade: mid-size → compact with a shorter `max_tokens` → template plus the retrieved excerpt.
3. Drop work whose client has disconnected.
4. Log `shed_reason`.

Impact: RS and CH (goodput stays flat instead of collapsing). 1 pd.

T2-11. Hint self-consistency checked by sympy (trigger: quota headroom at demo load)

Self-consistency cut ChatGPT hint failures from 32% of problems to near 0% in algebra (Pardos & Bhandari, PLOS ONE 2024). In EdNova, the T1-7 claim checks are a cheaper, stronger verifier than voting. Generate one hint, check it, and regenerate at most once. Use 2-3 samples with a vote only for guiding-question text that can't be checked, and only when `remaining_requests` is above the reserve. 0.25 pd.

Tier 3: week-two upgrades if time remains

T3-1. Buggy-rule CAS diagnosis (1.5-2 pd; RL and pedagogy; the best pedagogy-per-effort item). Write a recursive differentiator that swaps in known wrong rules:

- power rule: `POW_NO_DECREMENT`, `POW_NO_COEFF`

- constants: `CONST_KEPT`
- product rule: `PRODUCT_AS_PRODUCT` ((uv)' = u'v')
- quotient rule: `QUOTIENT_SIGN`, `QUOTIENT_NO_SQUARE`
- chain rule: `CHAIN_NO_INNER`
- trig: `SIGN_TRIG`

Precompute each exercise's buggy derivatives, singly and in pairs. When a learner's wrong answer is sympy-equivalent to one of them, return that bug's template feedback. Otherwise localize the wrong term by comparing term by term, or detect a constant-factor or sign slip. A comparable "model backtracking" diagnoser for quadratic equations agreed with teachers on 112 of 115 steps in about 0.35 s each (van der Hoek et al. 2025). Chain-rule error categories come from a small classroom study (Mahlan et al. ⚠, n = 22). Regression test: it must reproduce the 15 items' stored common mistakes.

T3-2. Deterministic tutor-move state machine (1 pd). Implement `next_move(state, diagnosis) -> Move` covering: affirm plus a "why" question; targeted correction for a known bug; hint level + 1; ask the learner to rewrite an unparseable answer; refuse "just give me the answer" before attempt 3; after hint 3 plus one more miss, show the full worked solution and then an isomorphic variant. The LLM writes at most 2 sentences for the chosen move. Bridge showed that an explicit decision step (error type → strategy → intention) helped GPT-4, but Llama-2 gained little from expert decisions (Bridge, NAACL 2024), so make the decision in code. Log MathDial's Telling@k and Success@k (MathDial).

T3-3. Warmed concept-explanation bank (L3 cache) during idle quota (2-3 pd for top-N concepts). Key = (concept_id, lang, level, prompt_version, kb_version), used for "explain concept" intents only and never personalized. Generate only when the last 60 s used fewer than 12 of 20 requests, with one background worker so a slot always stays free. Admit an entry only if it passes the T1-7 checks and language-ID. Store provenance with each entry. Apple's Krites puts a static vetted tier in front of a dynamic cache (Krites ⚠, judge simulated by an oracle), and preloading gold FAQs raised hit rates to 68-85% while lowering false positives (InfoQ case study ⚠, practitioner). Budget: 100 concepts × 5 languages × 3 levels = 1,500 requests, about 75 minutes of full quota, so warm the most-requested concepts first. Trigger: at least 30% of real explain requests fall in the warmed set.

T3-4. Offline doc2query, contextual headers and a bigger eval set (1.5 pd). For each chunk, generate 8-12 learner questions across FR, EN, MSA, Arabic-script Darija and Arabizi. Keep a question only if BM25 retrieves its own chunk first. Index the survivors in a lower-weight field. Filtering generated queries improved effectiveness by up to 16% and shrank the index by 33% (Doc2Query--). Contextual BM25 cut top-20 retrieval failures in Anthropic's tests (Contextual Retrieval). Grow the eval set to 60-100 real learner questions with a frozen 50% test split, and

compare runs with ranx and Fisher's randomization test (ranx). With 15 queries, one query is 6.7 points of hit@1. Adoption bar: cross-language hit@1 ⩾ 7/8 with no regressions.

T3-5. Canonical-intent cache key (L2) in shadow mode (1-2 pd). Key = (task, top-1 concept_id when the retrieval margin is high, sorted multiset of numbers and math tokens, lang, level, versions). Log would-be hits without serving them for 50 or more labelled samples. Go live only if precision is ⩾ 98% and the gain is ⩾ 5 points (LLMSigCache's zero-false-hit result depended on exact signature matching (LLMSigCache)).

T3-6. Fault-injection fake LiteLLM server and throttled evals (1 pd; proves RS). Reproduce exact P7 bodies and headers: key-quota 429 with `retry-after: 60`, cooldown 429/503 with 5, parallel cap, fractional and HTTP-date Retry-After, a 90 s 504, empty `choices`, and silent truncation. Toxiproxy works at TCP level only and can't emit 429s (Toxiproxy). Assertions: no unhandled exceptions, ⩽ 1.1 attempts per request, zero calls during a stated Retry-After, and a degraded result with a reason for every refusal. Route promptfoo or Inspect evals through EdNova's gatekeeper at background priority. Their defaults would flood a 2-concurrent key: Inspect retries indefinitely and promptfoo retries 4 times (Inspect, promptfoo).

T3-7. Per-learner fair queuing (0.5-1 pd). Keep strict priority between classes, and within a class schedule by deficit round-robin or a Virtual Token Counter keyed by learner, where cost = input tokens + 2 × output tokens (VTC, OSDI 2024). Allow one queued interactive request per learner, with the newest message replacing the older one. Trigger: a test with one learner sending 5 messages per minute alongside 9 normal learners raises the others' p95. Show Jain's fairness index before and after.

T3-8. Staged progress UX with a calibrated ETA range (0.5-1 pd). Progress indicators make users willing to wait about 3× longer (NN/g), and explained, bounded waits feel shorter (Maister). In one VR study, "natural" fillers beat a generic processing icon (IVA study). Show truthful stages from real engine states, an ETA range computed as max(concurrency term, rate term) plus any active cooldown, and a "quick hint now" button that triggers the downgrade path. Log the predicted and actual wait, and tune the padding until at least 90% of actual waits fall under the shown upper bound.

T3-9. Dialect checker and Arabizi variants (1-1.5 pd). CAMeL Tools' MADAR-26 dialect identifier labels Tunis and Sfax and runs on CPU, but not on Windows (CAMeL DID docs). Score only the Arabic-only spans of curated or generated Darija. GlotLID has no Latin-script Tunisian label, so keep rule-based Arabizi detection: digits inside words plus markers, after GlotLID rules out confident French or English (GlotLID). TArC pairs Arabizi tokens with Arabic-script CODA* forms (TArC); mine those pairs for glossary spelling variants. Its license is unstated, so use it only for internal rule derivation.

T3-10. Sympy variant generator (1-2 pd). Templates are parameterized by skill, sampled with a seed, and solved by sympy. "Ugly" items are rejected, as are parameter sets where two bugs

give the same f' (following the MBT recommendation). Hints are generated from the rule skeleton, and mistakes come from T3-1. This enables isomorphic retries and post-tests (CBIT ⚠, abstract only).

Sixteen techniques to skip

Technique	Why it doesn't fit EdNova
LLMLingua / LLMLingua-2	End-to-end gains appear mainly beyond about 5k tokens, compression added overhead below about 4k tokens on vLLM, and few-shot classification dropped by up to 52% (Prompt Compression in the Wild). It also needs an extra 1.5-3 GB CPU model, and no Arabic or Arabizi evaluation exists (BDTechTalks ⚠). Deterministic trimming (T1-4) gets the benefit.
RM3 pseudo-relevance feedback	Scored below plain BM25 on MRR@10 (15.8 vs 18.4) (query2doc baselines). With 15 chunks the feedback set is most of the corpus, so expansion drifts and weakens the coverage gate.
Query2doc / HyDE at query time	One more LLM call (over 2 s each in the paper, more on CPU), and a 1.3B model added only +0.8 (query2doc). Doc2query (T3-4) moves the benefit offline.
SPLADE / learned sparse	Efficient SPLADE checkpoints are English-only (naver). BGE-M3's sparse mode reached only 23.5 Recall@100 cross-lingual for Arabic, versus 71.1 for dense (M3).
BM25 variant switching / bm25s	Variants span 39.5-39.9 nDCG@10 (BM25S), and speed is not a problem at 15 chunks.
SelfCheckGPT / semantic entropy	SelfCheckGPT used 20 samples per query (SelfCheckGPT). Semantic entropy costs 5-10× compute, and its cheap probe variant needs hidden states (SEPs). Sympy plus rule checks are cheaper and stronger for math.
Trained routers (RouteLLM mf/sw_ranking/bert, Hybrid LLM, FrugalGPT scorer, PORT, the full AutoMix POMDP)	They need embeddings (RouteLLM's need an OpenAI key) or labelled quality-gap data, and were trained on English chat with GPT-4-class pairs (RouteLLM, survey). AutoMix's verifier spends 8 samples per query.
Verbalized confidence from 3B models	gemma-1.1-2b's ECE was 0.42, with confidence almost unrelated to correctness (Yang et al.). Recalibration does not improve error-detection AUROC (certified deferral ⚠, abstract only).
Atlas-Chat	Moroccan Darija models (Atlas-Chat) would reinforce ديال and بزاف, the exact drift to remove.

Technique	Why it doesn't fit EdNova
`no_repeat_ngram` decoding	Best raw loop score, but −8 MMLU and −11 GSM8K on a 70B model (Weidmann et al. ⚠). Math has to repeat n-grams like "= 2x".
Embedding semantic cache / LiteLLM `redis-semantic`	Needs an embedding model (LiteLLM semantic cache). Correct and incorrect hits have overlapping similarity scores (vCache), and "2x+3=7" vs "2x+3=9" is a guaranteed false hit. Reconsider only if an embedding endpoint appears, with a number-equality gate and shadow mode.
MinHash / SimHash near-duplicate cache	No production or academic evidence was found; the gain over T1-6 and T3-5 would be marginal.
Merging classify with a free-form hint on 3B; JSON-wrapped explanations	Small models lose accuracy on free-form multi-task prompts (MTI Bench), and JSON-forced reasoning collapses (Let Me Speak Freely?).
⩽8B LLM-as-judge for pedagogy	Llama-3.1-8B and Prometheus2 correlated mostly negatively with humans on pedagogical dimensions (MRBench), and the best BEA 2025 systems reached only 0.58-0.72 macro-F1 (BEA 2025). Use small judges only for narrow yes/no checks validated against human labels.
Langfuse self-hosting; Toxiproxy for 429s	Langfuse needs Postgres, ClickHouse, Redis and S3 (Langfuse); JSONL plus an optional Phoenix container is enough (Phoenix). Toxiproxy is TCP-only (Toxiproxy).
Cross-encoder reranker, Tangent-style formula search, skeleton screens	A reranker adds CPU latency with no benefit at 15 chunks. Full formula search is overkill for a handful of derivative rules (Tangent-L). Skeleton screens scored worst on perceived wait in an informal study (Viget ⚠).

Claims to treat as hypotheses until tested

Several load-bearing claims in this plan rest on weak sources. The Monday probes exist mainly to settle the first group.

Claim	Status	Consequence if wrong
LiteLLM `ollama_chat` maps `frequency_penalty` straight to `repeat_penalty`	Read in two mirrored older branches; 1.103 not inspected (mirror); the backend and provider prefix are unknown	T1-1 still helps through its temperature profiles; P2 decides
DRY gives −47% loops with no capability loss	Aug 2026 preprint read via an alphaXiv summary, not peer-reviewed (alphaXiv)	T2-3 only; measure the loop rate yourself
Ollama has logprobs on its OpenAI API since 0.12.11	Third-party release-notes mirror; the official docs still list logprobs as unsupported (Cloudron)	P4 decides
llama.cpp `/tokenize`; `dry_*` accepted on `/v1`	Docs mirrors and commit titles; the README was blocked for one researcher	P3 and P11 decide
17.7× prefix-cache prefill speedup	Secondary book chapter, GPU example (Leanpub)	Don't claim a CPU gain until P9 shows one
Classify is output-bound; requests/turn ~2.08 → ~1.6	Estimates from informal CPU benchmarks and an assumed traffic mix	P12 and Monday logs give the real numbers
LiteLLM v3 fixed-window limiter with `retry-after = 60` on FelCloud	Source read from a mirror; the deployed limiter version is unknown	P7 and P8 decide
"Lost in the middle" figures	Read via secondary summaries (Weaviate)	Small effect with only 2 chunks anyway
Tunisian marker lists, Arabizi digit table, Tunisian math term list, CAMeL license	General knowledge, no fetched source	Native-speaker and CNP-textbook validation before shipping T1-9
Krites gains; banking and VentureBeat cache numbers; Tian Pan practices	Oracle-simulated judge; practitioner write-ups	They motivate T3-3 and T3-5 only, both gated by shadow mode
Chain of Draft, certified deferral, CBIT, Daheim et al.	Abstract-only or preprint	Used only for optional or skip decisions

Claim	Status	Consequence if wrong
CPU tok/s figures; skeleton-screen study; CHI 2026 latency result (news coverage)	Informal or secondary	Used only for sizing and UX framing

Conclusion

The research changes where EdNova's remaining risk sits. It is less in the model and more in the plumbing between EdNova and the model: a sampler parameter that may be inverted in transit, a truncation edge that is invisible until a canary finds it, and a refusal signal (`Retry-After: 5`) that describes the shared backend rather than EdNova's key. All three can be settled on Monday at a cost of a few hundred requests, and each fix is a small change in code that already exists. The pattern is the same across tiers: wherever an LLM call can be replaced by code, sympy, a header or a rule, the replacement wins on all five judged metrics at once, because every request avoided also improves latency, quota headroom and resilience.

For the Phase 1 write-up, the strongest framing is to describe EdNova as a verifier-centred cascade rather than a set of separate guards. Rules and sympy decide, small models narrate, the mid-size model is spent only where a check fails, and every decision is logged with its cause. The literature supports this shape well (PAL, AutoMix, Bastani's guardrails and the routing survey all point at it), and for a CPU-only, 20-requests-per-minute deployment it is the only shape that scales. The techniques on the skip list fail because each one assumes spare calls, spare context or an embedding model that EdNova does not have.